TRIBHUVAN UNIVERSITY

INSTITUTE OF ENGINEERING

BE Programme | Year III / Part II

EMBEDDED SYSTEM (CT 655)

QUESTION BANK WITH ANSWERS

Chapter 1: Introduction to Embedded System

COURSE SYLLABUS AT A GLANCE

Unit	Topic	Hours	Marks
1	Introduction to Embedded System	3	4
2	Hardware Design Issues	4	8
3	Software Design Issues	6	8
4	Memory	5	8
5	Interfacing	6	8
6	Real-Time Operating System (RTOS)	8	12
7	Control System	3	8
8	IC Technology	3	8
9	Microcontrollers in Embedded Systems	3	8
10	VHDL	4	8
	Total	**45**	**80**

Chapter 1 Topics (3 Hours | 4 Marks):

- 1.1 Embedded Systems Overview
- 1.2 Design Metrics (NRE cost, unit cost, performance, power, size, flexibility, time-to-market)
- 1.3 Features and Characteristics of Embedded Systems
- 1.4 Classification of Embedded Systems (by generation, complexity, trigger)
- 1.5 Embedded System vs General-Purpose Computer
- 1.6 Applications of Embedded Systems
- 1.7 Hardware and Software in Embedded Systems
- 1.8 Typical Embedded System Components (block diagram)

Chapter 1: Introduction to Embedded System

Q1. Define embedded system and briefly explain the purpose of embedded system.	2082 Kartik (Back) [1+3]

ANSWER

Definition:

An embedded system is a dedicated computer system designed to perform one or a few specific functions, often under real-time constraints, tightly integrated into the device it controls. It combines hardware and software to perform a specific task, and unlike a general-purpose computer, it is not user-reprogrammable in general operation.

Examples: washing machine controller, digital camera, ATM, anti-lock braking system (ABS), pacemaker, traffic light controller.

Purpose of Embedded Systems:

- Control and Automation: Automatically control physical processes (e.g., washing machine cycles, engine management).
- Real-Time Processing: Respond to inputs within strict time deadlines (e.g., ABS must respond within milliseconds).
- Data Acquisition and Processing: Collect sensor data and process it (e.g., industrial temperature monitoring).
- Communication: Manage data transmission between devices (e.g., router firmware, smartphone baseband).
- Power Efficiency: Perform tasks with minimal energy consumption (critical for battery-powered devices).
- Safety and Reliability: Provide reliable, fail-safe control for life-critical systems (pacemaker, aircraft controls).

Q2. What is an embedded system? Explain any three design metrics with an example.	2081 Chaitra (Regular) [1+3]

ANSWER

An embedded system is a special-purpose computing system embedded inside a larger product to control it or assist its function. It is constrained in terms of power, cost, size, and performance. (See Q1 for full definition.)

Design Metrics:

Design metrics are criteria used to evaluate and compare different implementations of an embedded system. Three key metrics:

1. NRE Cost (Non-Recurring Engineering Cost): The one-time cost to design and develop the system — engineering man-hours, CAD tools, mask sets for custom ICs. Example: Designing a custom ASIC costs $1M NRE but produces chips at $0.50 each. For 10M units, NRE is amortized to $0.10/unit. For low volumes, a microcontroller solution with $0 NRE but $5/chip may be preferred.

2. Power Consumption: The energy consumed per unit time (watts). Critical for battery-operated devices. Example: A wearable fitness tracker must run for 7 days on a 100mAh battery — this constrains all design choices including processor speed, sleep modes, and peripheral usage.

3. Performance: The speed at which the system can complete its task. Measured as throughput (tasks/second) or latency (time per task). Example: A video codec embedded system must decode 30 frames per second (30 fps) HD video — this sets the minimum performance requirement on the chosen processor.

Q3. Why is power an important design metric in embedded systems? Explain.	2080 Chaitra (Regular) [1+3]

ANSWER

Power as a Critical Design Metric:

Power consumption is one of the most important design metrics in embedded systems for the following reasons:

- Battery Life: Most embedded systems are battery-powered (smartphones, wearables, remote sensors, medical implants). Excessive power consumption shortens battery life, making the product impractical. Example: A pacemaker must run for 7–10 years on its internal battery.
- Heat Dissipation: Power is dissipated as heat. Excessive heat can damage components, reduce reliability, or require costly cooling solutions. In compact embedded devices, heat dissipation is a major design challenge.
- Operating Cost: In always-on embedded systems (IoT sensors, smart meters), power translates directly to electricity cost over the product's operational lifetime.
- Regulatory Compliance: Many consumer electronics are subject to energy efficiency regulations (e.g., ENERGY STAR, EU ErP Directive). Embedded systems must meet legal power limits.
- Thermal Safety: In body-worn medical devices or devices near flammable materials, excessive heat generation is a safety hazard.
- Size and Weight: Higher power consumption requires larger batteries, adding weight and size — critical constraints in mobile and wearable devices.

Power management techniques used in embedded systems:

- Dynamic voltage and frequency scaling (DVFS): Reduce voltage/frequency when full performance is not needed.

- Sleep/idle modes: Put the processor and peripherals into low-power states when inactive.
- Clock gating: Disable clocks to unused modules to prevent switching power dissipation.
- Use of low-power hardware: Choose MCUs with low active and standby currents (e.g., ARM Cortex-M0+).

Q4. Define design metric and explain various design metrics used in embedded system design.	**2078 Chaitra (Regular) | 2074 Jestha (Back) [4]**

ANSWER

Design Metric: A design metric is a measurable property of a system implementation that is used to evaluate and compare design choices against the product's requirements.

Design Metrics in Embedded Systems:

1. NRE Cost (Non-Recurring Engineering Cost): One-time cost to design and develop the system. Includes engineering labor, CAD tool licenses, ASIC mask sets. Amortized over number of units. Prefer low NRE for low-volume products.

2. Unit Cost: Per-unit manufacturing cost — silicon area, package, PCB, assembly. Dominant metric for high-volume consumer products.

3. Performance: Speed of task completion. Expressed as throughput (tasks/sec), latency (time/task), or MIPS. Must meet real-time deadlines.

4. Power Consumption: Energy consumed per unit time. Critical for battery-operated and thermally-constrained devices. Measured in watts or milliwatts.

5. Size: Physical dimensions and weight of the system. Critical for wearables, implantables, aerospace systems.

6. Flexibility: Ability to adapt to changing requirements after deployment (firmware updates, configurable parameters). Higher flexibility = easier maintenance.

7. Time-to-Market (TTM): How quickly the product can be designed and deployed. Faster TTM provides competitive advantage. General-purpose processors have shorter TTM than custom ASICs.

8. Safety: Probability that the system does not cause harm. Critical for medical, automotive, and aerospace applications. Measured by failure rate (FIT — Failures In Time).

9. Correctness: Probability that the system meets its specification exactly. Verified through simulation, formal verification, and testing.

10. Maintainability: Ease of modifying the system after deployment to fix bugs or add features. Programmable systems (GPP, MCU) are more maintainable than custom hardware.

Q5. What are the features of an embedded system? Give examples.	2079 Chaitra (Regular) [4]

ANSWER

Features/Characteristics of Embedded Systems:

1. Single-Functioned: Designed to perform one specific function repeatedly. Example: A washing machine controller only manages wash cycles.

2. Tightly Constrained: Subject to strict design constraints — limited power, cost, size, and performance budget. Unlike desktop PCs, there is no room for overprovisioning.

3. Reactive and Real-Time: Must respond to external events (sensor inputs, interrupts) within defined time deadlines. Hard real-time: missing a deadline causes system failure (ABS brakes). Soft real-time: occasional misses are tolerable (video streaming).

4. Microprocessor/Microcontroller Based: Contains a programmable processing core. This makes them flexible — behavior can be changed by software update.

5. Connected to Hardware: Directly interfaces with sensors, actuators, displays, and communication modules through I/O pins (GPIO, ADC, DAC, UART, SPI, I2C).

6. Low Power Consumption: Designed to operate on battery for extended periods. Employ sleep modes, clock gating, and low-voltage operation.

7. Dedicated Operating System or No OS: Many run a simple bare-metal firmware loop or a lightweight RTOS (FreeRTOS, uC/OS). Complex embedded systems may run Linux.

8. Reliability and Robustness: Must operate continuously and correctly in harsh environments (temperature extremes, vibration, EMI). Often include watchdog timers to recover from faults.

9. Small Size and Low Cost: Highly integrated SoCs (System-on-Chip) integrate CPU, RAM, Flash, and peripherals on one chip to minimize PCB area and cost.

10. Domain-Specific: Optimized for a specific application domain. DSPs for signal processing, microcontrollers for control, GPUs for graphics.

Q6. How can we differentiate embedded systems from general-purpose computers? Explain with examples.	2080 Ashwin (Back) [4]

ANSWER

Property	Embedded System	General-Purpose Computer
Purpose	Single/few dedicated functions	Runs any application
Programmability	Fixed application, not user-reprogrammable	Fully user-programmable
Cost	Very low (high-volume production)	Moderate to high
Power	Very low (mW to W range)	High (tens to hundreds of watts)
Size	Compact, often chip-scale	Larger (laptops, desktops)

Operating System	None / RTOS / lightweight OS	Full OS (Windows, Linux, macOS)
Performance	Domain-optimized, often lower MHz	High performance, high MHz/GHz
Interface	Direct hardware I/O (GPIO, ADC, PWM)	Standard peripherals (USB, HDMI)
Reliability	Must be extremely reliable	Occasional reboots tolerated
Examples	ABS, pacemaker, washing machine	PC, laptop, server

Q7. What do you mean by embedded system? What are its essential features? Explain with examples.	**2081 Ashwin (Back) [1+3]**

ANSWER

Embedded System (see Q1 for definition). Essential features (see Q5 for complete list).

Three most essential features with examples:

1. Single-Functioned: An embedded system is purpose-built. A digital camera's embedded system only manages image capture, autofocus, image processing, and storage — it does not run spreadsheets or browse the internet.

2. Real-Time Operation: A car's embedded engine control unit (ECU) must respond to sensor inputs (throttle position, O2 sensor, crankshaft position) within microseconds to maintain fuel efficiency and emissions compliance. Missing these deadlines causes engine malfunction.

3. Tightly Constrained: A smart glucose monitor worn by diabetic patients must fit in a coin-sized patch, consume <1mA average current for 14-day battery life, and cost <$5 in materials. These simultaneous constraints drive every design decision.

Q8. Draw the block diagram of a digital camera as an embedded system and explain each block.	**2079 Ashwin (Back) [4]**

ANSWER

Block Diagram of Digital Camera as an Embedded System:

```
┌──────────────────────────────────────────────────────────────────────────┐
│                          DIGITAL CAMERA                                  │
│                                                                          │
│  [Lens+Aperture] → [CCD/CMOS Sensor] → [Analog Front End (AFE)]          │
│                                                  ↓                       │
│                                            [ADC]                         │
│                                                  ↓                       │
│  [Shutter/Zoom    [Image Signal      [Main Processor  [DRAM              │
│   Motor Driver] ← Processor (ISP)] ← (ARM/DSP)]    → Frame Buffer]       │
│                          ↓                  ↓                            │
│                   [JPEG Encoder]      [LCD Display]                      │
│                          ↓                  ↑                            │
```

```
│  [Flash Memory / SD Card]          [User Input: Shutter/Buttons]   │
│                                                                     │
│  [Battery Management] ← [Power Supply Unit]                        │
│  [USB / HDMI Interface]                                            │
└────────────────────────────────────────────────────────────────────┘
```

Block Descriptions:

- Lens + Aperture: Focuses light onto the image sensor. Zoom lens controlled by embedded motor drivers.
- CCD/CMOS Image Sensor: Converts light into analog electrical signals (photoelectric effect). Millions of photosites = megapixels.
- Analog Front End (AFE) + ADC: Amplifies and converts the analog sensor output to digital data.
- Image Signal Processor (ISP): Hardware accelerator for demosaicing, white balance, noise reduction, and sharpening — processes raw sensor data into viewable image.
- Main Processor (ARM/DSP): Runs the camera firmware — manages state machine (idle/preview/capture/review), JPEG encoding, user interface, file system.
- DRAM Frame Buffer: Temporary storage for full-resolution images during processing.
- JPEG Encoder (HW or SW): Compresses the image to JPEG format for storage.
- Flash Memory / SD Card: Non-volatile storage for captured photos and camera firmware.
- LCD Display: Shows preview and captured images to the user.
- User Input (Shutter, Buttons): Generates interrupts to the main processor on user actions.
- Battery Management IC: Monitors battery voltage/current, controls charging, and implements low-battery protection.

Q9. What are the common characteristics of embedded systems? Explain digital camera as an embedded system. What are the kernel services provided by the operating system in embedded systems?	**2072 Ashwin (Back) [4+4]**

ANSWER

Common Characteristics: (See Q5 for full list of 10 characteristics.)

Digital Camera as Embedded System: (See Q8 for full block diagram and explanation.)

Kernel Services in Embedded Operating Systems:

An embedded OS kernel provides the following core services:

- Task/Process Management: Creates, schedules, suspends, and terminates tasks. Implements priority-based preemptive scheduling (RTOS). Example: FreeRTOS task scheduler.
- Memory Management: Allocates and deallocates heap memory. Simple embedded kernels use fixed-size memory pools to avoid heap fragmentation. Provides memory protection (MPU) in advanced systems.
- Inter-Task Communication: Message queues, semaphores, mutexes, event flags, and mailboxes allow tasks to exchange data and synchronize safely.
- Interrupt Management: Registers Interrupt Service Routines (ISRs). Manages interrupt priority and nesting. Defers long processing to tasks via signaling from ISR.
- Timer Services: Software timers for periodic callbacks, timeouts, and delays. Hardware timer abstraction.
- Device Drivers / I/O Management: Abstracts hardware peripherals (UART, SPI, I2C, ADC) behind a standard API. Manages buffering, DMA, and interrupt-driven I/O.
- Power Management: Controls sleep/wake cycles, clock scaling, and peripheral power gating to minimize energy consumption.

Q10. Define and explain embedded system. Classify embedded systems with examples.	**2079 Jestha (Back) [3+1]**

ANSWER

Embedded System: (See Q1.)

Classification of Embedded Systems:

Embedded systems can be classified in multiple ways:

A) By Complexity / Size:

- Small-scale: 8/16-bit MCU, simple sensors, no OS. Example: toy controller, temperature sensor.
- Medium-scale: 16/32-bit MCU, RTOS, networking. Example: industrial PLC, home router.
- Large-scale/Sophisticated: 32/64-bit processor, full OS (Linux), complex peripherals. Example: smartphone, automotive infotainment.

B) By Trigger / Behavior:

- Event-Triggered: Executes tasks in response to external events/interrupts. Example: keyboard controller (responds to keypress).
- Time-Triggered: Executes tasks periodically based on a timer. Example: data logger sampling sensor every 100ms.

C) By Real-Time Nature:

- Hard Real-Time: Missing a deadline causes catastrophic failure. Example: anti-lock braking system, aircraft flight control.

- Soft Real-Time: Occasional deadline misses degrade quality but are tolerable. Example: video streaming, digital audio player.
- Firm Real-Time: Deadline misses are tolerable but valueless — late results are discarded. Example: online stock trading system.

Q11. Define Embedded System and classify the embedded systems by generations.	**2072 Magh (Back) [4+4]**

ANSWER

Embedded System: (See Q1.)

Classification by Generations:

Generation	Era	Key Technology	Characteristics	Examples
1st Generation	1970s	8-bit microprocessors (8080, Z80)	No OS, simple I/O, single task, assembly language	Early calculators, industrial controllers
2nd Generation	1980s	8/16-bit MCUs (8051, PIC, HC11)	On-chip peripherals, simple RTOS possible, C language	Embedded controllers, automotive ECUs
3rd Generation	1990s	16/32-bit DSPs + RISC MCUs (ARM7, MIPS)	DSP integration, RTOS, networking, higher performance	Routers, mobile phones, digital cameras
4th Generation	2000s	SoC (System-on-Chip), 32-bit ARM	Full OS (Linux, WinCE), multimedia, wireless, rich UI	Smartphones, tablets, set-top boxes
5th Generation	2010s–present	Multi-core SoC, IoT, AI accelerators	Heterogeneous computing, AI/ML inference, cloud connectivity	Smart devices, autonomous vehicles, wearables

Q12. What is an embedded system? List its types. Why is it hard to define embedded system?	**2073 Bhadra (Regular) [3+1+4]**

ANSWER

Embedded System: (See Q1.)

Types of Embedded Systems (See Q10 and Q11 for full classifications):

- By complexity: Small-scale, Medium-scale, Large-scale
- By real-time: Hard, Soft, Firm
- By trigger: Event-triggered, Time-triggered
- By generation: 1st through 5th generation

Why Embedded Systems are Hard to Define:

- No single definition covers all: A simple 8-bit MCU in a toy and a 64-bit multi-core SoC in a smartphone are both 'embedded systems,' yet they differ enormously in complexity, capability, and design.
- Overlap with general-purpose: Modern smartphones and tablets are highly programmable and run general-purpose OSes — yet they are considered embedded systems because they are integrated into a specific product for a specific purpose.
- Rapidly evolving technology: New processor architectures, IoT platforms, and AI accelerators constantly blur the boundaries. Yesterday's 'general-purpose computer' becomes today's embedded platform.
- Domain-specific definitions: Automotive engineers, medical device developers, and consumer electronics designers each use slightly different definitions appropriate to their context.
- Hard vs. soft constraints: Some embedded systems have hard real-time constraints; others do not. Some are resource-constrained; others have gigabytes of RAM. No single property is universal.

Q13. Explain the hardware components of an embedded system with a neat block diagram.	**Model Question | 2075 Bhadra (Back) [4]**

ANSWER

Hardware Components of an Embedded System:

```
┌──────────────────────────────────────────────────────────────────────────┐
│                    EMBEDDED SYSTEM HARDWARE                              │
│                                                                          │
│  INPUT DEVICES        PROCESSOR CORE          OUTPUT DEVICES             │
│  ┌──────────┐        ┌──────────────────┐     ┌──────────────┐           │
│  │ Sensors  │──────►│ CPU / MCU / DSP  │──►│ Actuators  │           │
│  │ Switches │        │                  │     │ Displays   │           │
│  │ ADC      │        │  ┌────────────┐  │     │ DAC/PWM    │           │
│  └──────────┘        │  │  On-chip   │  │     └──────────────┘           │
│                      │  │ Peripherals│  │                                │
│  MEMORY              │  │ (UART,SPI,│  │     COMMUNICATION              │
│  ┌──────────┐        │  │  I2C,ADC)  │  │     ┌──────────────┐           │
│  │ Flash/ROM│◄──────►│  └────────────┘  │◄──►│ UART/SPI   │           │
│  │ SRAM/RAM │        └──────────────────┘     │ I2C/CAN    │           │
│  └──────────┘                                 │ Ethernet   │           │
│                                               └──────────────┘           │
│  POWER SUPPLY                                                            │
│  ┌──────────────────────────────────────────────────────────┐           │
│  │ Battery / Regulator / PMIC / Power Management            │           │
│  └──────────────────────────────────────────────────────────┘           │
└──────────────────────────────────────────────────────────────────────────┘
```

- Processor Core (CPU/MCU/DSP): The brain of the system. Executes instructions from Flash memory. Controls all operations.
- Memory: Flash/ROM stores the program code (non-volatile). SRAM/RAM stores variables, stack, and runtime data (volatile).
- Input Devices: Sensors (temperature, pressure, motion), switches, and ADCs convert physical quantities to digital signals.
- Output Devices: Actuators (motors, solenoids, relays), displays (LCD, LED), and DAC/PWM outputs.
- On-Chip Peripherals: UART (serial communication), SPI/I2C (sensor buses), ADC (analog input), PWM (motor/LED control), Timers.
- Communication Interfaces: UART, SPI, I2C, CAN (automotive), Ethernet, Wi-Fi, Bluetooth for system-to-system communication.
- Power Supply: Regulated power for all components. Battery + PMIC (Power Management IC) for portable systems.

Q14. What is an RTOS? Why is an RTOS used in embedded systems? Differentiate between hard real-time and soft real-time systems.	**Model Question | 2076 Chaitra [4]**

ANSWER

RTOS (Real-Time Operating System):

An RTOS is a lightweight operating system kernel designed specifically for embedded systems that must respond to events within guaranteed, deterministic time bounds. Unlike a desktop OS (which optimizes for average throughput), an RTOS optimizes for worst-case response time (latency).

Why RTOS is Used in Embedded Systems:

- Task scheduling: Manages multiple concurrent tasks (sensors, UI, communication) with priority-based preemptive scheduling.
- Deterministic timing: Guarantees maximum latency for interrupt response and task switching — essential for real-time control.
- Resource management: Provides semaphores, mutexes, and message queues for safe inter-task communication.
- Modularity: Separates concerns — each task handles one function (sensor, control, display), making the system easier to develop and test.
- Examples: FreeRTOS (open-source), uC/OS-III (commercial), Zephyr, VxWorks, QNX.

Hard Real-Time vs Soft Real-Time Systems:

Property	Hard Real-Time	Soft Real-Time
Deadline	Absolute — must always be met	Statistical — occasional misses OK

Consequence of miss	System failure (catastrophic)	Degraded quality (tolerable)
Examples	ABS brakes, pacemaker, flight control	Video streaming, music player, GUI
OS requirement	Deterministic RTOS essential	RTOS preferred but not critical
Scheduling	Fixed-priority preemptive	May use best-effort scheduling

Q15. Write short notes on: (a) Microprocessor vs Microcontroller (b) Sensor and Actuator in Embedded Systems (c) Watchdog Timer	**Model Question | 2077 Magh (Back) [2+2+2+2]**

ANSWER

(a) Microprocessor vs Microcontroller:

Property	Microprocessor	Microcontroller
Integration	CPU only (no RAM/ROM/peripherals)	CPU + RAM + Flash + peripherals on one chip
External components	Needs external RAM, ROM, I/O chips	Mostly self-contained
Cost	Higher (system cost)	Lower (single chip)
Power	Higher	Lower
Application	Desktop PCs, workstations	Embedded control (appliances, IoT)
Examples	Intel Core i7, AMD Ryzen	STM32, PIC, AVR, 8051

(b) Sensor and Actuator in Embedded Systems:

Sensor: A device that converts a physical quantity (temperature, pressure, motion, light) into an electrical signal that the embedded system can read. Examples: thermistor (temperature → resistance), accelerometer (acceleration → voltage), PIR sensor (infrared → digital pulse).

Actuator: A device that converts an electrical signal from the embedded system into a physical action. Examples: DC motor (PWM signal → rotation), solenoid valve (digital signal → fluid flow), LED/buzzer (digital → light/sound).

(c) Watchdog Timer:

A watchdog timer (WDT) is a hardware timer built into most microcontrollers that resets the system if the software fails to 'kick' (reload) it within a defined period.

- Purpose: Automatically recovers the system from software hangs, infinite loops, or crashes — critical for unattended embedded systems.
- Operation: Software must write a specific value to the WDT register periodically (e.g., every 100ms). If the write doesn't happen (software crash), the WDT times out and triggers a hardware reset.
- Example: An industrial sensor node in a remote location uses a WDT to recover from firmware crashes without human intervention.

Q16. What do you understand by design challenges in embedded systems? Explain the challenges related to hardware-software co-design.	2078 Jestha (Back) | 2075 Magh (Back) [4+4]

ANSWER

Design Challenges in Embedded Systems:

- Simultaneous Constraints: Must meet multiple conflicting objectives simultaneously — high performance AND low power AND low cost AND small size.
- Resource Limitations: Extremely limited memory (KB, not GB), limited processing power, and limited I/O.
- Real-Time Requirements: Must guarantee response times — requires careful analysis of worst-case execution time (WCET) for every task.
- Reliability and Safety: Embedded systems often control safety-critical processes. A bug in a pacemaker or car brake system can be fatal.
- Long Product Lifetimes: Industrial embedded systems may need to operate for 20+ years without software updates.
- Verification and Testing: Testing embedded systems on actual hardware is time-consuming. Hardware may not be available during software development.
- Heterogeneous Components: Modern embedded systems include CPUs, DSPs, FPGAs, and ASICs in one product — integrating these is complex.

Hardware-Software Co-Design:

Hardware-software co-design is the simultaneous design of hardware and software components of an embedded system to best meet overall system requirements.

- Partitioning: Deciding which functions are implemented in hardware (fast, low-latency, but inflexible) vs. software (flexible, updatable, but slower).
- Co-Simulation: Simulating hardware and software together before any hardware exists — uses tools like SystemC, VHDL/Verilog simulation with processor models.
- Interface Design: Defining precise hardware-software interfaces (memory maps, interrupts, DMA channels) that both teams develop to.
- Trade-off Analysis: Evaluating cost, performance, and power implications of hardware vs. software implementations.
- Challenges: Hardware and software teams work in parallel with different tools and languages. Changes to one affect the other. Integration testing is complex.

Summary: Topics Covered in Chapter 1

S.N.	Topic	Exam(s) Asked	Marks
1	Definition and Purpose of Embedded System	2082, 2081, 2080, 2079, 2078, 2073, 2072 (multiple exams every year)	1+3 to 4
2	Design Metrics (all 10 metrics)	2081 Chaitra, 2078 Chaitra, 2074 Jestha, 2072 Magh	3–8
3	Power as Design Metric	2080 Chaitra	1+3
4	Features/Characteristics of Embedded Systems	2079 Chaitra, 2081 Ashwin, 2077 Magh	4
5	Embedded System vs General-Purpose Computer	2080 Ashwin, 2076 Chaitra, 2073 Bhadra	4
6	Classification by Generation	2072 Magh, 2073 Bhadra, 2079 Jestha	4+4
7	Classification by Real-Time / Trigger	2079 Jestha, 2076 Chaitra	4
8	Digital Camera Block Diagram	2079 Ashwin, 2072 Ashwin	4
9	Kernel OS Services in Embedded Systems	2072 Ashwin	4
10	Hardware Components / Block Diagram	2075 Bhadra, Model Q	4
11	RTOS / Hard vs Soft Real-Time	2076 Chaitra, 2077 Magh, Model Q	4
12	Microprocessor vs Microcontroller	2077 Magh, Model Q	2–4
13	Sensor and Actuator	2077 Magh, Model Q	2–4
14	Watchdog Timer	Multiple	2
15	Design Challenges / HW-SW Co-design	2078 Jestha, 2075 Magh	4+4
16	Why Embedded Systems are Hard to Define	2073 Bhadra	4